



Canteen Management Portal

Name: Bhashkar Kumar Gupta
Enrolment: xxxxxxxxxxxxxxxx
Course: BCA
Regional centre code: 400
Study Centre: 0748464
Subject: BCSP-064

Project guide's Name:
Mohit Sharma
Qualification: MCA
(5 Years experience in IT)

1. Introduction and Overview

The Canteen Management Portal is designed to streamline the daily operations of a canteen. This portal enables users to place orders, manage inventory, handle venue bookings, and receive feedback, all in one centralized platform. It also provides an organized, efficient system for canteen staff and administrators to manage various tasks, such as tracking inventory, processing orders, and handling user requests.

Purpose: The primary reason for building this portal is to automate and simplify canteen management tasks, both for users (customers) and the staff. Traditionally, managing orders, inventory, and venues involves manual tracking and can lead to errors, delays, and inefficiencies. By creating this portal, we aim to:

- Provide users with a more convenient way to place and track their orders.
- Allow cooks to manage menus and inventory effectively.
- Help administrators manage venue bookings, announcements, and user accounts with ease.
- Ensure seamless integration between the canteen's daily operations and its long-term planning.
- This portal was chosen for development due to its potential to reduce operational complexity and improve user experience for all stakeholders involved.



2. Objectives of the Canteen Management Portal

The Canteen Management Portal is developed with the following key objectives in mind:

Centralized Operations: To provide a unified platform where users, cooks, and administrators can perform daily canteen operations. This includes ordering food, managing inventory, and handling venue bookings all in one place.

Automation: To automate and simplify several routine tasks in canteen management. By replacing manual processes with automated systems, this platform aims to reduce errors and improve operational efficiency.

User Convenience: To enhance the user experience by providing an easy and convenient way to browse menus, place orders, and track order history. Users can also provide feedback and rate the service, helping to improve the overall quality of service over time.

Inventory Management: To allow cooks and administrators to effectively track inventory levels, including automatic updates after orders are placed. This ensures that the inventory is up to date, reducing instances of stock shortages or wastage.

Menu Management: To enable cooks and administrators to manage the menu, update daily or weekly items, and ensure that users are aware of the latest available dishes.

Venue Management: To streamline the process of booking venues within the canteen, enabling users to reserve spaces for events. Administrators can easily approve, reject, or manage these bookings.

Feedback System: To provide a feedback mechanism where users can share their experiences and rate the food and venues. This helps the canteen management improve service quality based on real user data.



Security and Access Control: To ensure secure operations through authentication and role-based access control. Users, cooks, and administrators have different levels of access based on their roles, ensuring that sensitive operations are restricted to authorized personnel.

Efficient Communication: To offer a platform for sending announcements, managing user accounts, and efficiently handling canteen-related requests.



3. Tools/Environment Used

For building the **Canteen Management Portal**, the following tools and technologies were utilized:

Backend: Node.js with PostgreSQL

The backend was built using **Node.js** with **PostgreSQL** as the database. A set of dependencies were used to manage various aspects of the system such as authentication, database operations, security, and scheduling tasks. Below is a breakdown of the key backend dependencies:

1. **bcryptjs:**

- **Purpose:** Used for hashing and verifying passwords.
- **Functionality:** Ensures secure storage of user passwords by hashing them before saving them to the database. It is also used during user login to verify the hashed password.

2. **cors:**

- **Purpose:** Cross-Origin Resource Sharing middleware.
- **Functionality:** Enables the backend to communicate with the frontend, which may be hosted on a different domain, by allowing requests from specified origins.

3. **dotenv:**

- **Purpose:** Loads environment variables from a .env file.
- **Functionality:** Ensures that sensitive configuration data (e.g., database connection strings, JWT secrets) can be securely loaded into the Node.js application without hardcoding them into the source code.

4. **express:**

- **Purpose:** The web framework used to create APIs.
- **Functionality:** Provides a simple and flexible way to handle HTTP requests and define routes for the various operations in the application (e.g., handling orders, managing menus, managing users).



5. **express-rate-limit:**

- **Purpose:** Rate limiting for requests.
- **Functionality:** Prevents brute force attacks and abuse by limiting the number of requests a user can make to the backend API within a certain time window.

6. **helmet:**

- **Purpose:** Security middleware.
- **Functionality:** Adds security-related HTTP headers to protect the app from well-known web vulnerabilities like cross-site scripting (XSS), clickjacking, etc.

7. **jsonwebtoken:**

- **Purpose:** For managing JSON Web Tokens (JWT).
- **Functionality:** Used for user authentication and authorization. It generates and verifies JWT tokens to ensure only authenticated users can access certain resources.

8. **node-cron:**

- **Purpose:** Scheduling cron jobs.
- **Functionality:** Allows scheduling of regular tasks such as inventory checks or automated email reminders. For example, it could be used to check inventory levels and trigger low stock alerts.

9. **nodemailer:**

- **Purpose:** Email sending.
- **Functionality:** Sends email notifications, such as low inventory alerts or user notifications, through predefined SMTP configurations.

10. **pg & pg-hstore:**

- **Purpose:** PostgreSQL client for Node.js and Sequelize.
- **Functionality:** Handles database operations. pg connects to the PostgreSQL database, and pg-hstore is a helper for managing key-value pairs in the database.



11. **sequelize:**

- **Purpose:** Object-Relational Mapper (ORM) for Node.js.
- **Functionality:** Manages database interactions in an object-oriented manner. It simplifies database queries and relationships by mapping JavaScript objects to database tables. Sequelize is used for database operations such as creating, reading, updating, and deleting records in PostgreSQL.

12. **nodemon** (development dependency):

- **Purpose:** Development tool for automatic server restarts.
- **Functionality:** Automatically restarts the Node.js server whenever a change in the code is detected during development, improving development efficiency.

Frontend: React with Workbox and Axios

The frontend of the Canteen Management Portal was built using **React.js**, a popular JavaScript framework for building user interfaces. The following are the main dependencies used in the frontend:

1. **React:**

- **Purpose:** Core library for building user interfaces.
- **Functionality:** Handles the rendering of components, state management, and user interactions.

2. **Axios:**

- **Purpose:** HTTP client for making API requests.
- **Functionality:** Simplifies communication between the frontend and backend by making asynchronous HTTP requests to the backend APIs (e.g., fetching menu items, placing orders).

3. **Bootstrap & React-Bootstrap:**

- **Purpose:** Frontend UI framework.
- **Functionality:** Provides pre-built responsive design components, such as buttons, forms, and modals, making the application visually appealing and mobile-friendly.



4. **React-Router-Dom:**

- **Purpose:** Routing library for React.
- **Functionality:** Handles navigation between different pages of the web application, such as the menu, order history, and venue booking pages.

5. **JWT-Decode:**

- **Purpose:** Token decoding utility.
- **Functionality:** Decodes JWT tokens stored on the client side to extract user information (e.g., roles and permissions), enabling role-based access control.

6. **React-Hook-Form:**

- **Purpose:** Form validation library.
- **Functionality:** Manages and validates form inputs, such as login, registration, and order forms, improving the user experience with error handling and validation.

7. **React-Toastify:**

- **Purpose:** Notification library.
- **Functionality:** Provides non-intrusive notification pop-ups to inform users about the success or failure of actions, such as placing an order or submitting feedback.

8. **Workbox Libraries:**

- **Purpose:** Progressive Web App (PWA) capabilities.
- **Functionality:** Workbox libraries provide advanced caching strategies, offline access, background synchronization, and network request routing. These are essential for making the application function smoothly even when there is limited network connectivity.
- **Modules Used:**
 - **workbox-core:** Provides core functionality for building a PWA.
 - **workbox-precaching:** Handles caching of static assets (HTML, CSS, JS) during the service worker's install event.
 - **workbox-strategies:** Defines how requests are handled (e.g., cache first, network first).
 - **workbox-background-sync:** Ensures requests (such as submitting feedback or orders) are sent once the network is restored.



- **workbox-routing:** Configures the routing of network requests within the service worker.

9. **React-Datepicker:**

- **Purpose:** Date selection component.
- **Functionality:** Simplifies the selection of dates for booking venues or setting deadlines for orders.

10. **React-Icons:**

- **Purpose:** Icon library for React.
- **Functionality:** Provides scalable icons for use across the application, improving its visual appeal.

11. **Date-Fns:**

- **Purpose:** Date utility library.
- **Functionality:** Simplifies operations related to date manipulation, such as formatting or comparing dates, making it useful for tasks like displaying order history or venue bookings.

12. **Testing Libraries:**

- **@Testing-Library/React** and related packages: Used for writing unit and integration tests for React components.

These tools and libraries helped create a robust, efficient, and scalable solution for managing canteen operations, ensuring that the portal is both secure and user-friendly while leveraging modern development practices like PWA features.



4. Analysis Document:

SRS (Software Requirement Specification):

1.1 Product Perspective:

The Canteen Management Portal is a web-based application that integrates various canteen-related tasks into a single system. It automates order placement, menu updates, and inventory tracking, reducing manual intervention. The system's architecture is based on a RESTful API built with Node.js, using PostgreSQL for the backend and React.js for the frontend.

1.2 Product Features:

Key features include:

- **User Authentication and Role-based Access:** Users can register, log in, and access features based on their roles (user, admin, cook, root).
- **Menu Management:** Admins and cooks can manage menu items, update availability, and set daily or weekly menus.
- **Order Management:** Users can place and track orders, and the system will manage payments and update inventory accordingly.
- **Inventory Management:** Cooks and admins can manage stock, and the system will send low-stock alerts.
- **Venue Booking:** Users can book venues for events, with approval from the admin.
- **Feedback and Ratings:** Users can provide feedback on menu items and venues.
- **Announcements:** Admins can post announcements for all users.



1.3 User Classes and Characteristics:

- **Users:** Can browse the menu, place orders, rate food, book venues, and view order history.
- **Cooks:** Can manage menus, view orders, and update inventory.
- **Admins:** Have access to all features, including user management, venue approvals, inventory updates, and announcements.
- **Root User:** Has full control over the system and can manage admins and cooks.

1.4 Operating Environment:

- **Backend:** Node.js with Express.js, PostgreSQL for the database, running on a Linux server.
- **Frontend:** React.js, Bootstrap for UI design.
- **Development Tools:** VSCode for development, Postman for API testing, and Git for version control.

1.5 Assumptions and Dependencies:

- Users have internet access to interact with the web-based system.
- The backend server and PostgreSQL database are hosted and accessible.
- Third-party libraries like JWT for authentication and Nodemailer for emails are correctly configured.



Specific Requirements

2.1 Functional Requirements:

User Authentication and Role-based Access:

- The system must allow users to register, log in, and reset passwords.
- Role-based access control must be implemented:
 - **Users** can view menus, place orders, rate items, and book venues.
 - **Admins** can manage menus, users, inventory, and announcements.
 - **Cooks** can view orders, manage the menu and inventory.
 - **Root users** can manage all aspects of the system, including creating admin and cook accounts.

Menu Management:

- Admins and cooks must be able to add, update, or delete menu items.
- Users should be able to view the menu and filter by category.
- The menu must support both daily and weekly configurations.

Order Management:

- Users can place orders, which will update inventory levels based on ingredients.
- Admins and cooks can view, accept, reject, or complete orders.
- The system must deduct inventory for each accepted order.

Inventory Management:

- Cooks and admins can add, update, or delete inventory items.
- The system must alert admins when inventory falls below a threshold level.

Venue Booking:

- Users can book venues by providing details such as date, time, and purpose.
- Admins can approve, reject, or update booking statuses.

Feedback and Ratings:

- Users can provide feedback on menu items and rate their experiences.
- Admins can view all feedback and use it for service improvements.



Announcements:

- Admins can post announcements that will be visible to all users.

2.2 Non-Functional Requirements:**Security:**

- **Authentication:** The system will use JWT tokens for secure authentication.
- **Authorization:** Role-based access control must ensure that only authorized users can perform certain actions (e.g., only admins can manage users).
- **Password Storage:** User passwords must be hashed before being stored in the database.

Performance:

- The system should handle multiple concurrent user sessions efficiently.
- Database queries should be optimized to reduce response time.

Usability:

- The interface must be user-friendly, with clear navigation for different roles.
- Pages should load quickly, and all interactions must provide user feedback (e.g., notifications).

2.3 Database Requirements:

- The database must store user information, orders, menu items, inventory levels, feedback, venue bookings, and announcements.
- It must ensure **referential integrity** between tables, particularly for orders, menus, and inventory items.



Entity-Relationship (ER) diagrams:

The **ER diagram** of the Canteen Management Portal represents the relationships between various entities such as **Users**, **Menu Items**, **Orders**, **Inventory**, **Venues**, and more. This model shows how these entities interact with one another, defining the structure of the database and ensuring data integrity across the system.

Key Entities and Relationships:

1. Users:

- Represents users of the system, including regular users, admins, cooks, and root users.

Relationships:

- **One-to-Many with Orders:** Each user can place multiple orders.
- **One-to-Many with Announcements:** A user (admin/cook) can create many announcements.
- **One-to-Many with Feedbacks:** A user can provide multiple feedback entries.
- **One-to-Many with VenueBookings:** A user can book multiple venues.
- **One-to-Many with VenueFeedbacks:** A user can give feedback for multiple venues.

2. Menu Items:

- Represents food items available on the menu.

Relationships:

- **One-to-Many with OrderItems:** A menu item can be part of multiple orders.
- **One-to-Many with Ratings:** Each menu item can have multiple ratings.
- **Many-to-Many with InventoryItems (through MenuItemIngredients):** Menu items can have multiple ingredients, and inventory items can be used in multiple menu items.
- **One-to-Many with WeeklyMenu:** A menu item can appear in multiple weekly menus.



3. Orders and Order Items:

- **Orders:** Represent orders placed by users, containing one or more menu items.
- **OrderItems:** Represents individual items within an order.

Relationships:

- **One-to-Many with OrderItems:** Each order can contain multiple order items.
- **One-to-Many with Users:** Each user can place multiple orders.

4. Inventory Items:

- Represents stock items in the canteen, including ingredients and utensils.

Relationships:

- **Many-to-Many with Menu Items (through MenuItemIngredients):** Inventory items can be associated with multiple menu items, ensuring proper tracking of ingredients for food preparation.

5. Announcements:

- Represents public announcements made by admins or cooks.

Relationships:

- **One-to-Many with Users:** Each user (admin/cook) can create multiple announcements.

6. Feedback and Ratings:

- **Feedback:** Represents feedback provided by users about their experiences with menu items or venues.
- **Ratings:** Represents ratings provided by users for specific menu items.

Relationships:

- **One-to-Many with Users:** Users can provide feedback or ratings for menu items.
- **One-to-Many with Menu Items:** Each menu item can have multiple ratings.
- **One-to-Many with Venues:** Users can provide feedback for venues.



7. Venues and Venue Bookings:

- **Venues:** Represents physical venues within the canteen, available for booking.
- **VenueBookings:** Represents bookings made by users for canteen venues.

Relationships:

- **One-to-Many with Users:** Users can book multiple venues.
- **One-to-Many with Venues:** Each venue can have multiple bookings.

8. Weekly Menu:

- Represents the menu items assigned to specific days of the week.

Relationships:

- **One-to-Many with Menu Items:** Each menu item can be part of multiple weekly menus.

Many-to-Many Relationships:

- **Menu Items and Inventory Items:** This relationship is managed through the **Menu_Item_Ingredients** table, which tracks the ingredients used in each menu item. Each menu item can require multiple inventory items (ingredients), and each inventory item can be used in multiple menu items.

Additional Considerations:

- **Unique Constraints:**
 - **Ratings:** Each user can only rate a menu item once (unique constraint on menu_item_id and user_id).
- **Status Tracking:**
 - **Orders** and **VenueBookings** have statuses to track their progress (e.g., pending, accepted, completed for orders and pending, approved, rejected for venue bookings).



ER Diagram: -



Table Structure:

Users (Users table)
id: Primary key, auto-incremented.
name: String, required.
employee_id: String, unique, required.
email: String, unique, required.
password_hash: String, required.
address: Text, optional.
contact_number: String, optional.
role: ENUM ('user', 'admin', 'cook', 'root'), default 'user'.
credit_balance: Decimal (10, 2), default 0.00.
created_at: Date, default current date.
updated_at: Date, auto-updated on modification.

Announcements (Announcements table)
id: Primary key, auto-incremented.
title: String, required.
content: Text, required.
created_by: Foreign key to Users (id).

Feedbacks (Feedbacks table)
id: Primary key, auto-incremented.
content: Text, required.
user_id: Foreign key to Users (id)

• Inventory Items (Inventory table)
id: Primary key, auto-incremented.
item_name: String, required.
quantity: Float, default 0.
item_type: String (e.g., 'ingredient', 'utensil').
comments: Text, optional.
updated_at: Date, auto-updated on modification.



Menu Items (Menu_Items table)
id: Primary key, auto-incremented.
name: String, required.
description: Text, optional.
price: Decimal (10, 2), required.
availability: ENUM ('available', 'out of stock'), default 'available'.
category: String, required.
rating: Decimal (2, 1), default 0.0.
total_ratings: Integer, default 0.

Order Items (Order_Items table)
id: Primary key, auto-incremented.
quantity: Integer, required.
price: Decimal (10, 2), required.
menu_item_id: Foreign key to Menu_Items (id).
order_id: Foreign key to Orders (id).

Orders (Orders table)
id: Primary key, auto-incremented.
total_amount: Decimal (10, 2), required.
status: ENUM ('pending', 'accepted', 'rejected', 'completed'), default 'pending'.
created_at: Date, default current date.
updated_at: Date, auto-updated on modification.
user_id: Foreign key to Users (id)
created_at: Date, default current date.

Ratings (Ratings table)
id: Primary key, auto-incremented.
menu_item_id: Foreign key to Menu_Items (id).
user_id: Foreign key to Users (id).
rating: Integer (1-5), required.
Unique constraint on menu_item_id and user_id (one user can rate a menu item once).



Venue Bookings (Venue_Bookings table)
id: Primary key, auto-incremented.
booking_date: Date, required.
start_time: Time, required.
end_time: Time, required.
purpose: Text, optional.
status: ENUM ('pending', 'approved', 'rejected'), default 'pending'.
admin_comment: Text, optional.
user_id: Foreign key to Users (id).
venue_id: Foreign key to Venues (id).

Venues (Venues table)
id: Primary key, auto-incremented.
venue_name: String, required.
location: String, optional.
description: Text, optional.

Venue Feedbacks (VenueFeedbacks table)
id: Primary key, auto-incremented.
venue_id: Foreign key to Venues (id).
user_id: Foreign key to Users (id).
content: Text, required.

Weekly Menus (WeeklyMenu table)
id: Primary key, auto-incremented.
day: ENUM ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'), required.
menu_item_id: Foreign key to Menu_Items (id).



Menu Item Ingredients (Menu_Item_Ingredients table)
id: Primary key, auto-incremented.
quantity_required: Float, required.
menu_item_id: Foreign key to Menu_Items (id).
inventory_item_id: Foreign key to Inventory (id).

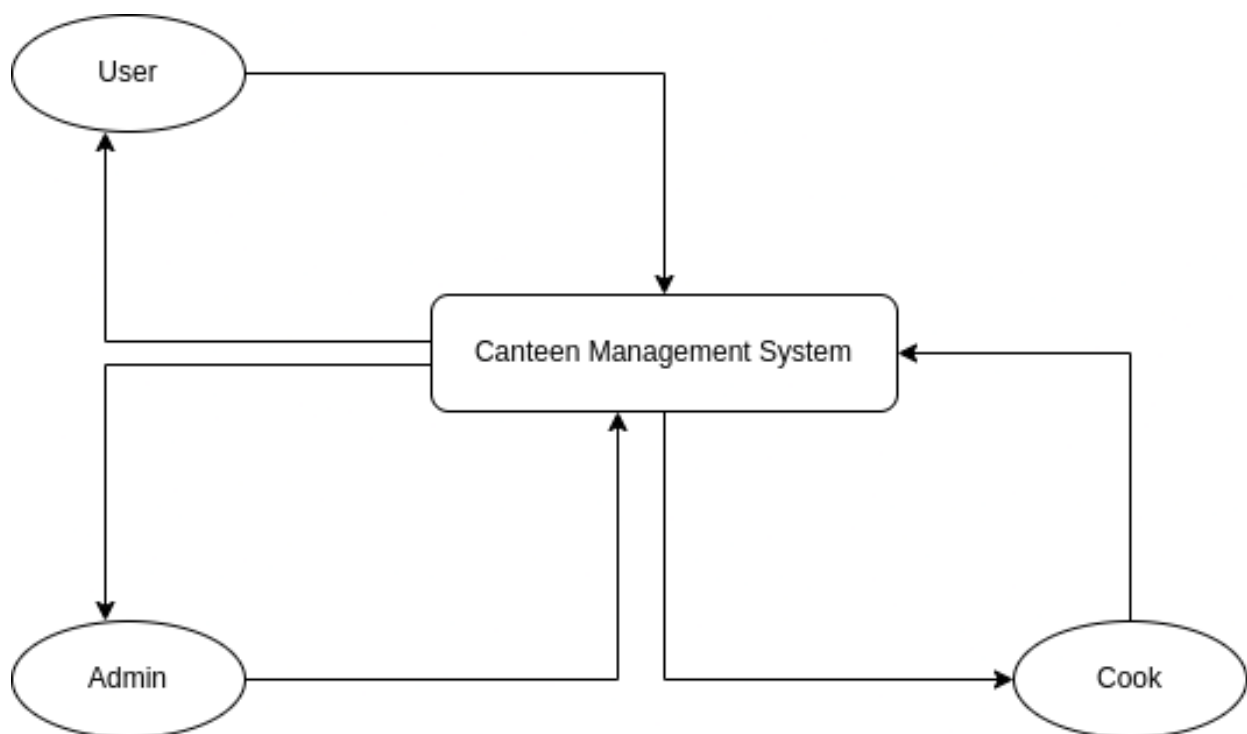


DFD (Data Flow Diagram)

Level 0 DFD (Context Diagram):

The **Level 0 DFD**, or the **Context Diagram**, provides a high-level overview of the Canteen Management Portal and its interactions with external entities. It illustrates how data flows between the **Canteen Management System** and key actors such as **Users**, **Admin**, and **Cook**.

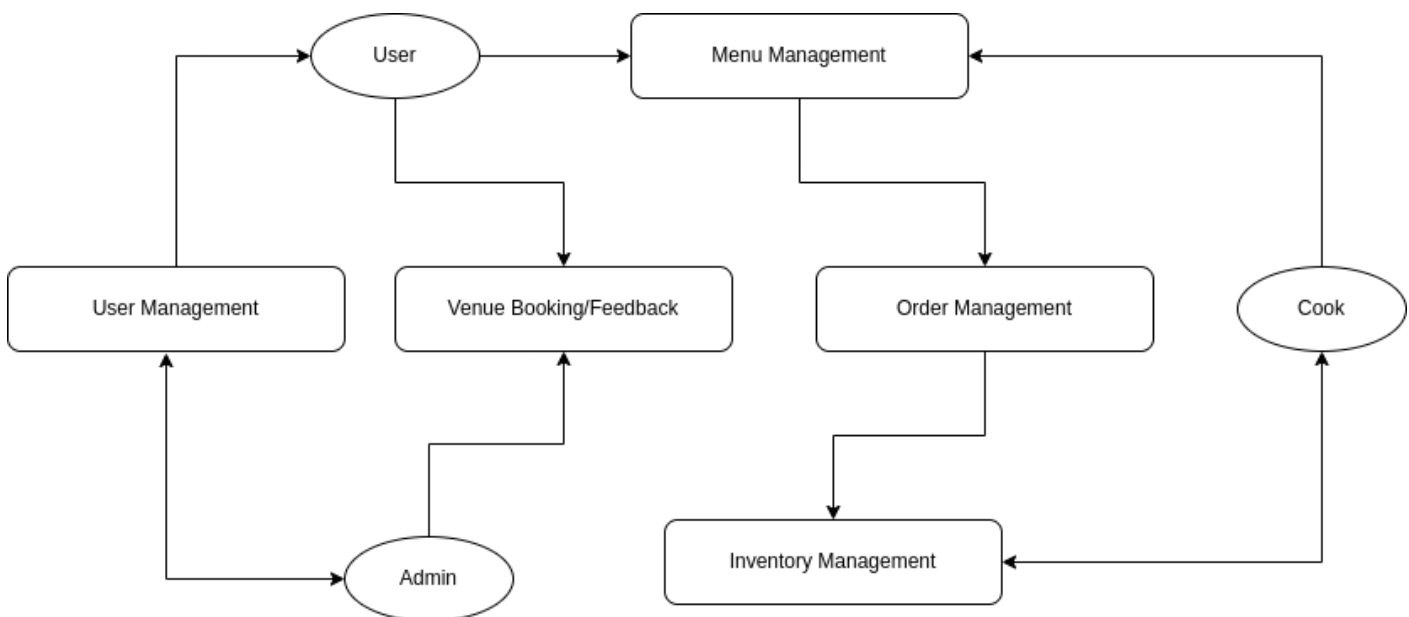
- **Users** interact with the system by placing orders, booking venues, and providing feedback.
- **Admins** manage users, venues, and inventory while ensuring that the system runs smoothly.
- **Cooks** update inventory, manage orders, and create or update menu items.
- The system acts as a mediator, handling the operations and ensuring data consistency.



Level 1 DFD (Main Processes):

The **Level 1 DFD** breaks down the system into its main functional components:

1. **User Management:** Handles user registration, login, and profile updates.
2. **Menu Management:** Allows cooks and admins to create, update, or delete menu items, which users can view and order from.
3. **Order Management:** Manages the process of users placing orders, cooks updating the order status, and users viewing their order history.
4. **Venue Booking:** Manages venue bookings, including booking requests by users, approval/rejection by admins, and users providing venue feedback.
5. **Inventory Management:** Handles adding, updating, and deleting inventory items, managed by both cooks and admins.
6. **Feedback and Ratings:** Users provide feedback and rate menu items or venues.



Level 2 DFDs (Detailed Processes):

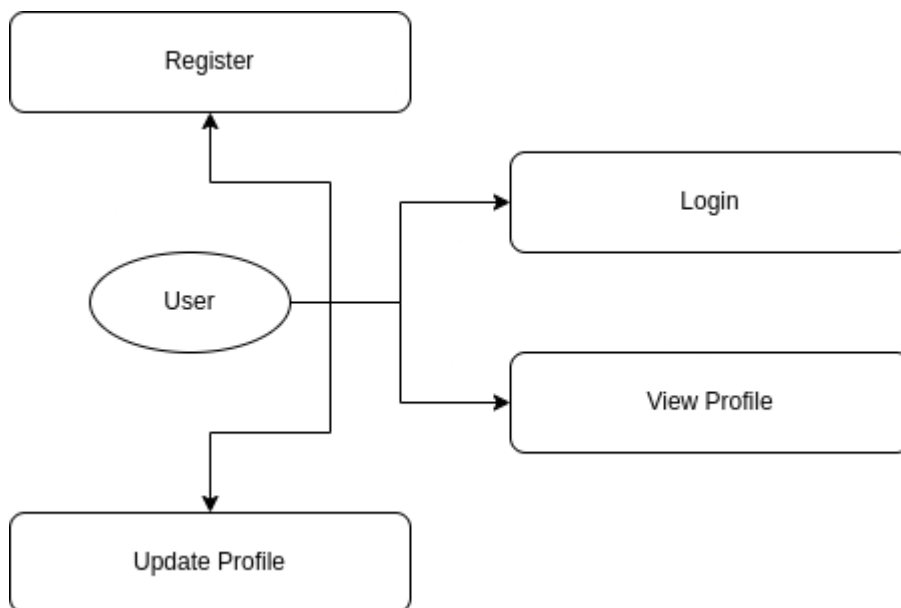
Each process from the Level 1 diagram is broken down further in Level 2 diagrams, showing detailed flows of data between system components.



1. User Management (Level 2 DFD):

This diagram breaks down the **User Management** process:

- **Registration:** Users provide their details to register. The system stores user data in the database and grants access based on the role (user, admin, cook).
- **Login:** The user logs in, and the system validates the credentials.
- **Profile Management:** Users can view and update their profiles.
- **Data Store:** All user information is securely stored in the user database.

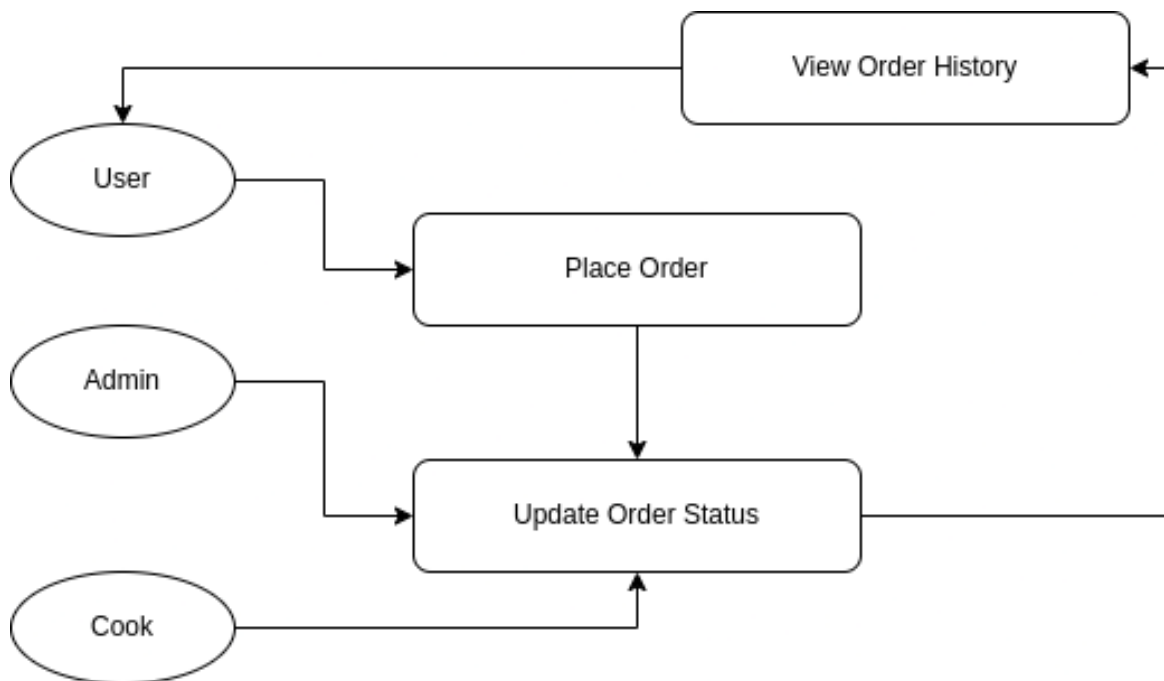


2. Order Management (Level 2 DFD):

This diagram explains the **Order Management** process:

- **Place Order:** The user selects items from the menu and places an order.
- **Update Order Status:** Cooks and admins can update the order status (pending, accepted, rejected, completed).
- **View Order History:** Users can view their past orders and check the status.
- **Data Store:** Order data is stored and linked with user information.

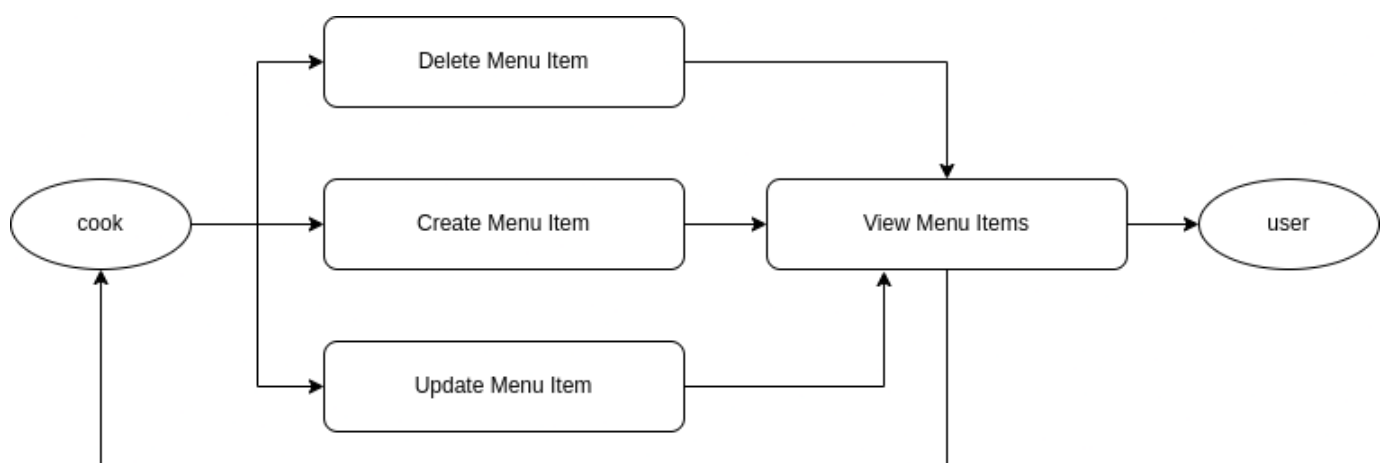




3. Menu Management (Level 2 DFD):

This DFD details the **Menu Management** process:

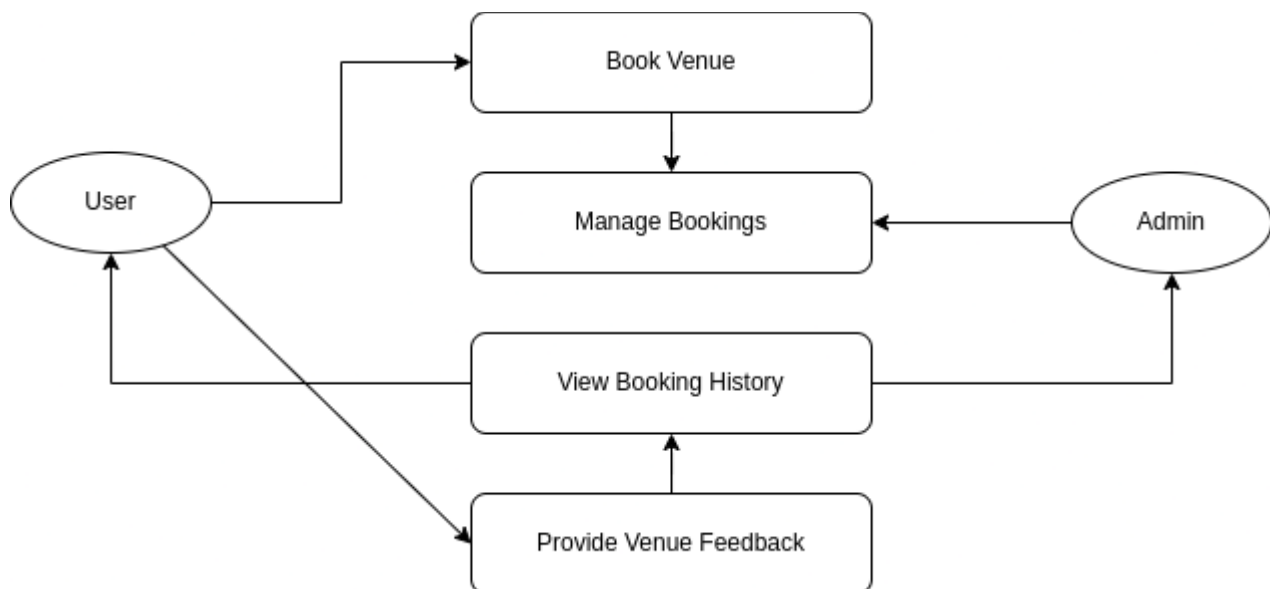
- **Create/Update/Delete Menu Items:** Cooks or admins can add new menu items, update existing ones, or delete items from the menu.
- **View Menu:** Users can view available menu items and place orders.
- **Data Store:** Menu items and their related information (name, price, availability, ratings) are stored in the system.



4. Venue Booking (Level 2 DFD):

This diagram outlines the **Venue Booking** process:

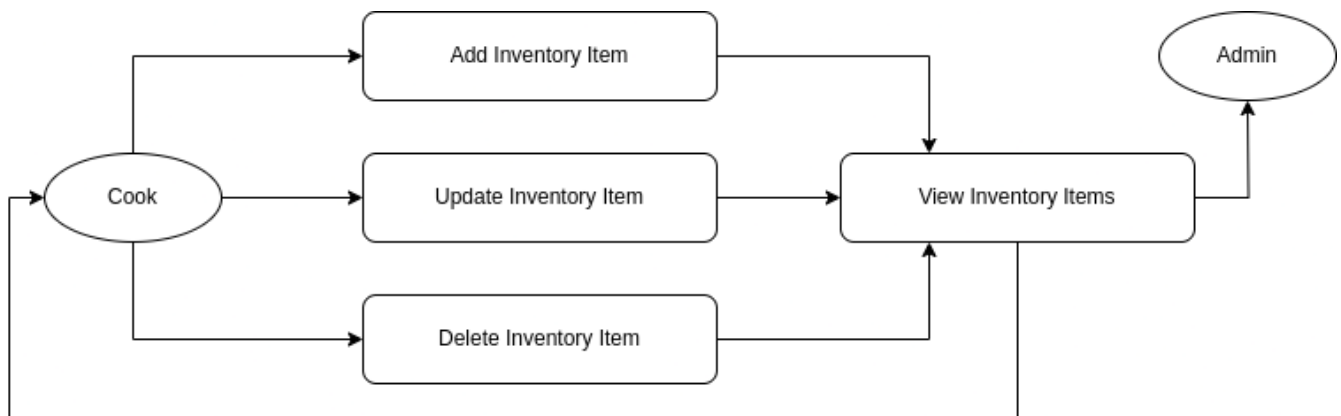
- **Book Venue:** Users submit venue booking requests with details such as date, time, and purpose.
- **Manage Bookings:** Admins review the booking requests and either approve or reject them. Users can view the status of their bookings.
- **Provide Feedback:** After the venue is used, users can submit feedback for the venue.
- **Data Store:** All venue booking and feedback data is stored and can be accessed by admins for future analysis.



5. Inventory Management (Level 2 DFD):

This diagram shows the detailed process of **Inventory Management**:

- **Add/Update/Delete Inventory Items:** Cooks and admins manage the inventory, including adding new items, updating quantities, or deleting items.
- **View Inventory:** Cooks and admins can view the inventory levels to ensure there is enough stock for future orders.
- **Data Store:** Inventory data is stored and linked with the menu items to track usage.



5. Design Document:

Modular design using MVC architecture:

Modular Design:

In this project, each feature is separated into individual modules (e.g., user management, order management, menu management, etc.), each of which consists of its **model**, **controller**, **routes**, and **middleware** where applicable. This modular design ensures that the functionality of each feature is encapsulated, allowing it to be developed and maintained independently without affecting other parts of the system.

The following are the main modules in the Canteen Management Portal:

1. **User Management:** Handles user registration, login, and profile management.
2. **Menu Management:** Manages the creation, deletion, and updating of menu items.
3. **Order Management:** Manages the placement of orders and updates the status of those orders.
4. **Venue Booking:** Handles booking of venues and related feedback.
5. **Inventory Management:** Manages canteen inventory and stock levels.
6. **Feedback and Ratings:** Allows users to provide feedback and rate menu items or venues.
7. **Announcements:** Allows administrators to post announcements.

Each module is organized into:

- **Models:** Represent the database schema (tables).
- **Controllers:** Handle business logic and communicate with the models to retrieve or modify data.
- **Routes:** Define the HTTP endpoints and methods to expose the API to external systems or the frontend.
- **Middleware:** Manages tasks such as authentication, authorization, and role-based access.



MVC Architecture in the Canteen Management Portal:

The system follows the **Model-View-Controller (MVC)** pattern, a design architecture that divides the application into three interconnected components:

1. Model (Database Interaction Layer):

- The **model** represents the data and business logic. It defines the structure of data and manages interactions with the database using **Sequelize ORM**.
- Each model corresponds to a table in the **PostgreSQL** database.
- Examples:
 - **User Model**: Manages users and their attributes (e.g., name, email, role).
 - **Order Model**: Represents the orders placed by users.
 - **MenuItem Model**: Contains menu items, their prices, and availability.
- **Flow**: When the controller needs data (e.g., fetching all users or placing an order), it interacts with the model to retrieve, create, update, or delete data from the database.

2. Controller (Business Logic Layer):

- The **controller** processes incoming requests from the client or frontend. It calls the appropriate model to retrieve or modify data and then returns a response.
- The controller does not interact directly with the database but instead works through the models.
- Example:
 - **Order Controller**: Handles creating an order by validating the request, checking inventory, and then using the Order model to store the order in the database.
- **Flow**: When a user places an order, the request is received by the controller. The controller verifies the order and checks the availability of menu items. Once validated, it interacts with the order and inventory models to update the database.



3. View (Frontend Layer):

- While this project does not include a traditional "View" in the backend (since the frontend is handled separately), the **view** in MVC typically represents what the user sees on the frontend. It consumes the data returned by the controllers in response to API calls.
- The **React.js frontend** interacts with the backend via HTTP requests, and the backend sends JSON responses that the frontend renders.

Data Flow :

1. Client Request:

- The process starts when the user (through the frontend) sends an HTTP request (e.g., placing an order, booking a venue, viewing the menu).
- Example Request: POST /api/orders (User places an order).

2. Router:

- The request is handled by the corresponding **route** based on the URL pattern and HTTP method.
- Example Route: /api/orders is defined in **orderRoutes.js**. This route directs the request to the **Order Controller**.

3. Controller:

- The **controller** processes the request. For example, in the case of an order, the **Order Controller** (orderController.js) is invoked.
- It performs validations (e.g., whether the user has enough credits, the availability of menu items) and processes business logic.
- The controller then interacts with the **models** to perform database operations.

4. Model:

- The **model** interacts with the **PostgreSQL** database through **Sequelize ORM**. It either retrieves data (e.g., available menu items) or modifies data (e.g., deducting ingredients from inventory after an order is placed).



- Example: The **Order Model** stores the new order and updates the user's order history.

5. Response:

- After the data is processed and stored in the database, the controller sends a response back to the client (e.g., a success message or the details of the placed order).
- Example Response: { "status": "success", "order": { "id": 123, "status": "pending", "items": [...] } }.

API Routes in the Canteen Management Portal:

The Canteen Management Portal utilizes various **RESTful API routes** to handle the interactions between the frontend and backend. These routes are organized into separate modules for ease of development and maintainability, each with specific responsibilities such as user management, order processing, menu management, etc.

1. User Management Routes

The **userRoutes.js** file handles user-related operations such as registration, login, profile management, and role-based user management.

- **POST /api/users/register**
 - **Description:** Registers a new user.
 - **Request Body:** { name, email, password, role }
 - **Response:** A success message and user details (without the password hash).
- **POST /api/users/login**
 - **Description:** Logs in a user and returns a **JWT** for authentication.
 - **Request Body:** { email, password }
 - **Response:** JWT token for accessing protected routes.



- **GET /api/users/profile**
 - **Description:** Retrieves the profile of the logged-in user.
 - **Authentication:** Yes, JWT required.
 - **Response:** User profile details.
- **POST /api/users/register/root**
 - **Description:** Allows the root user to register new root, admin, and cook users.
 - **Authentication:** Root user role required.
 - **Request Body:** { name, email, password, role }
 - **Response:** A success message and the new user's details.
- **GET /api/users/get-all-users**
 - **Description:** Fetches all users. This is restricted to root and admin roles.
 - **Authentication:** Yes, JWT required.
 - **Response:** List of all registered users.

2. Menu Management Routes

The **menuRoutes.js** file handles all operations related to menu items. This includes adding, updating, viewing, and deleting menu items.

- **GET /api/menu**
 - **Description:** Retrieves all available menu items.
 - **Authentication:** Yes.
 - **Response:** Array of menu items with details such as price, availability, and category.
- **POST /api/menu**
 - **Description:** Adds a new menu item. Only admins and cooks can add menu items.



- **Authentication:** Yes, Admin or Cook role required.
- **Request Body:** { name, description, price, category, availability }
- **Response:** A success message with the new menu item details.
- **PUT /api/menu/**
 - **Description:** Updates a menu item. Only admins and cooks can perform this action.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Request Body:** { name, description, price, category, availability }
 - **Response:** A success message with the updated menu item.
- **DELETE /api/menu/**
 - **Description:** Deletes a menu item.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Response:** A success message confirming the deletion.

3. Order Management Routes

The **orderRoutes.js** file manages all aspects of order placement, order history, and order status updates.

- **POST /api/orders**
 - **Description:** Places a new order.
 - **Authentication:** Yes, User role required.
 - **Request Body:** { menuItems: [{ menu_item_id, quantity }], total_amount }
 - **Response:** A success message with the order details.
- **GET /api/orders/my-orders**
 - **Description:** Retrieves the order history for the logged-in user.
 - **Authentication:** Yes, User role required.
 - **Response:** Array of past orders with details like status and items.



- **GET /api/orders**
 - **Description:** Retrieves all orders (for root, cook, or admin roles).
 - **Authentication:** Yes, Admin, Cook, or Root role required.
 - **Response:** List of all orders.
- **PUT /api/orders/status**
 - **Description:** Updates the status of an order (e.g., pending, accepted, completed).
 - **Authentication:** Yes, Admin or Cook role required.
 - **Response:** A success message with the updated order status.

API Routes in the Canteen Management Portal

The Canteen Management Portal utilizes various **RESTful API routes** to handle the interactions between the frontend and backend. These routes are organized into separate modules for ease of development and maintainability, each with specific responsibilities such as user management, order processing, menu management, etc.

Below is a breakdown of the main routes and their details, including HTTP methods, endpoints, and a description of their functions:

1. User Management Routes

The **userRoutes.js** file handles user-related operations such as registration, login, profile management, and role-based user management.

- **POST /api/users/register**
 - **Description:** Registers a new user.
 - **Request Body:** { name, email, password, role }
 - **Response:** A success message and user details (without the password hash).
- **POST /api/users/login**



- **Description:** Logs in a user and returns a **JWT** for authentication.
- **Request Body:** { email, password }
- **Response:** JWT token for accessing protected routes.
- **GET /api/users/profile**
 - **Description:** Retrieves the profile of the logged-in user.
 - **Authentication:** Yes, JWT required.
 - **Response:** User profile details.
- **POST /api/users/register/root**
 - **Description:** Allows the root user to register new root, admin, and cook users.
 - **Authentication:** Root user role required.
 - **Request Body:** { name, email, password, role }
 - **Response:** A success message and the new user's details.
- **GET /api/users/get-all-users**
 - **Description:** Fetches all users. This is restricted to root and admin roles.
 - **Authentication:** Yes, JWT required.
 - **Response:** List of all registered users.

2. Menu Management Routes

The **menuRoutes.js** file handles all operations related to menu items. This includes adding, updating, viewing, and deleting menu items.

- **GET /api/menu**
 - **Description:** Retrieves all available menu items.
 - **Authentication:** Yes.
 - **Response:** Array of menu items with details such as price, availability, and category.



- **POST /api/menu**
 - **Description:** Adds a new menu item. Only admins and cooks can add menu items.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Request Body:** { name, description, price, category, availability }
 - **Response:** A success message with the new menu item details.
- **PUT /api/menu/**
 - **Description:** Updates a menu item. Only admins and cooks can perform this action.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Request Body:** { name, description, price, category, availability }
 - **Response:** A success message with the updated menu item.
- **DELETE /api/menu/**
 - **Description:** Deletes a menu item.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Response:** A success message confirming the deletion.

3. Order Management Routes

The **orderRoutes.js** file manages all aspects of order placement, order history, and order status updates.

- **POST /api/orders**
 - **Description:** Places a new order.
 - **Authentication:** Yes, User role required.
 - **Request Body:** { menuItems: [{ menu_item_id, quantity }], total_amount }
 - **Response:** A success message with the order details.
- **GET /api/orders/my-orders**



- **Description:** Retrieves the order history for the logged-in user.
 - **Authentication:** Yes, User role required.
 - **Response:** Array of past orders with details like status and items.
 - **GET /api/orders**
 - **Description:** Retrieves all orders (for root, cook, or admin roles).
 - **Authentication:** Yes, Admin, Cook, or Root role required.
 - **Response:** List of all orders.
 - **PUT /api/orders/status**
 - **Description:** Updates the status of an order (e.g., pending, accepted, completed).
 - **Authentication:** Yes, Admin or Cook role required.
 - **Response:** A success message with the updated order status.
-

4. Inventory Management Routes

The **inventoryRoutes.js** file manages the addition, update, and deletion of inventory items.

- **GET /api/inventory**
 - **Description:** Retrieves all inventory items (e.g., ingredients and utensils).
 - **Authentication:** Yes, Admin or Cook role required.
 - **Response:** Array of inventory items with details like quantity, type, and name.
 - **POST /api/inventory**
 - **Description:** Adds a new inventory item.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Request Body:** { item_name, quantity, item_type, comments }
 - **Response:** A success message with the new inventory item details.
-



- **PUT /api/inventory/**
 - **Description:** Updates an inventory item.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Request Body:** { item_name, quantity, item_type, comments }
 - **Response:** A success message with the updated inventory item.
- **DELETE /api/inventory/**
 - **Description:** Deletes an inventory item.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Response:** A success message confirming the deletion.

5. Venue Booking Routes

The **venueBookingRoutes.js** file manages venue bookings by users, as well as booking approvals or rejections by admins.

- **POST /api/venue-bookings**
 - **Description:** Requests a venue booking.
 - **Authentication:** Yes, User role required.
 - **Request Body:** { booking_date, start_time, end_time, purpose }
 - **Response:** A success message with the booking details.
- **GET /api/venue-bookings/my-bookings**
 - **Description:** Retrieves the logged-in user's venue booking history.
 - **Authentication:** Yes, User role required.
 - **Response:** Array of venue bookings.
- **PUT /api/venue-bookings/status**
 - **Description:** Updates the status of a venue booking (approved, rejected).
 - **Authentication:** Yes, Admin role required.



- **Response:** A success message with the updated booking status.
- **GET /api/venue-bookings**
 - **Description:** Retrieves all venue bookings (admin only).
 - **Authentication:** Yes, Admin role required.
 - **Response:** Array of all venue bookings.

6. Feedback and Ratings Routes

The **feedbackRoutes.js** and **ratingRoutes.js** files manage the submission of feedback and ratings for menu items and venues.

- **POST /api/feedback**
 - **Description:** Submits feedback for the canteen's service.
 - **Authentication:** Yes, User role required.
 - **Request Body:** { content }
 - **Response:** A success message confirming feedback submission.
- **GET /api/feedback**
 - **Description:** Retrieves all feedbacks (for admin, cook, or root users).
 - **Authentication:** Yes, Admin, Cook, or Root role required.
 - **Response:** Array of feedbacks from all users.
- **POST /api/ratings**
 - **Description:** Submits a rating for a menu item.
 - **Authentication:** Yes, User role required.
 - **Request Body:** { menu_item_id, rating }
 - **Response:** A success message confirming the rating submission.
- **GET /api/ratings/menu/**
 - **Description:** Retrieves the ratings for a specific menu item.



- **Authentication:** Yes.
- **Response:** Array of ratings for the specified menu item.

7. Announcement Routes

The `announcementRoutes.js` file allows admins and cooks to post public announcements.

- **GET /api/announcements**
 - **Description:** Retrieves all announcements.
 - **Authentication:** Yes, any authenticated user.
 - **Response:** Array of announcements.
- **POST /api/announcements**
 - **Description:** Creates a new announcement.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Request Body:** { title, content }
 - **Response:** A success message with the announcement details.
- **PUT /api/announcements/**
 - **Description:** Updates an existing announcement.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Request Body:** { title, content }
 - **Response:** A success message with the updated announcement.
- **DELETE /api/announcements/**
 - **Description:** Deletes an announcement.
 - **Authentication:** Yes, Admin or Cook role required.
 - **Response:** A success message confirming the deletion.



Controllers in the Canteen Management Portal:

1. UserController:

- Manages user-related actions like registration, login, and profile updates.
- Key Methods:
 - `registerUser()`: Registers a new user.
 - `loginUser()`: Authenticates users and generates JWT.
 - `getUserProfile()`: Retrieves the profile of the logged-in user.

2. MenuController:

- Handles all menu-related operations such as creating, updating, and deleting menu items.
- Key Methods:
 - `getMenuItems()`: Fetches all available menu items.
 - `addMenuItem()`: Adds a new menu item.
 - `updateMenuItem()`: Updates an existing menu item.
 - `deleteMenuItem()`: Removes a menu item.

3. OrderController:

- Manages the order lifecycle, including order placement and status updates.
- Key Methods:
 - `placeOrder()`: Allows users to place an order.
 - `getOrderHistory()`: Retrieves the order history for the user.
 - `updateOrderStatus()`: Updates the order status (pending, completed, etc.).

4. InventoryController:

- Manages the inventory of the canteen (e.g., ingredients, utensils).
- Key Methods:
 - `getInventoryItems()`: Retrieves all inventory items.



- `addInventoryItem()`: Adds a new inventory item.
- `updateInventoryItem()`: Updates an existing inventory item.
- `deleteInventoryItem()`: Removes an inventory item.

5. **VenueBookingController:**

- Manages the venue booking process.
- Key Methods:
 - `requestVenueBooking()`: Allows users to book a venue.
 - `getUserVenueBookings()`: Retrieves a user's venue booking history.
 - `updateBookingStatus()`: Admin updates booking status (approved, rejected).

6. **FeedbackController:**

- Handles user feedback submission and retrieval.
- Key Methods:
 - `submitFeedback()`: Allows users to provide feedback.
 - `getAllFeedbacks()`: Retrieves all feedback for the admin or cook.

7. **RatingController:**

- Manages ratings for menu items.
- Key Methods:
 - `submitRating()`: Allows users to rate a menu item.
 - `getRatingsForMenuItem()`: Retrieves ratings for a specific menu item.

8. **AnnouncementController:**

- Manages the creation and viewing of announcements by admins and cooks.
- Key Methods:
 - `createAnnouncement()`: Allows admins or cooks to create an announcement.
 - `getAllAnnouncements()`: Retrieves all announcements for users.



- `updateAnnouncement()`: Updates an existing announcement.
- `deleteAnnouncement()`: Deletes an announcement.

Models in the Canteen Management Portal:

1. User Model:

- Represents user information.
- Key Attributes:
 - `name`, `email`, `password_hash`, `role`, `credit_balance`, `created_at`, `updated_at`.

2. MenuItem Model:

- Represents individual menu items.
- Key Attributes:
 - `name`, `description`, `price`, `availability`, `category`, `rating`, `total_ratings`.

3. Order Model:

- Represents an order placed by a user.
- Key Attributes:
 - `total_amount`, `status`, `created_at`, `updated_at`, `user_id`.

4. OrderItem Model:

- Represents individual items within an order.
- Key Attributes:
 - `quantity`, `price`, `menu_item_id`, `order_id`.

5. InventoryItem Model:

- Represents items in the inventory.
- Key Attributes:
 - `item_name`, `quantity`, `item_type`, `comments`, `updated_at`.



6. VenueBooking Model:

- Represents venue bookings made by users.
- Key Attributes:
 - booking_date, start_time, end_time, purpose, status, user_id, venue_id, admin_comment.

7. Feedback Model:

- Represents user feedback submissions.
- Key Attributes:
 - content, user_id, created_at, updated_at.

8. Rating Model:

- Represents ratings given by users to menu items.
- Key Attributes:
 - menu_item_id, user_id, rating.

9. Announcement Model:

- Represents announcements made by admins or cooks.
- Key Attributes:
 - title, content, created_by, created_at, updated_at.

10. MenuItemIngredient Model:

- Represents the relationship between menu items and ingredients.
- Key Attributes:
 - quantity_required, menu_item_id, inventory_item_id.

11. WeeklyMenu Model:

- Represents the menu items assigned to specific days of the week.
- Key Attributes:
 - day, menu_item_id.



8. Testing:

1. Cross-Browser Testing:

Testing the portal on multiple browsers ensures consistent user experience across platforms. The system was tested on:

- **Google Chrome:** Ensured smooth functionality of all features.
- **Mozilla Firefox:** Verified responsiveness and no compatibility issues.
- **Microsoft Edge:** Checked layout and UI rendering.
- **Safari (macOS):** Confirmed that the system works well without visual discrepancies.

Testing Steps:

- Checked the responsiveness of the interface (menus, orders, forms).
- Ensured compatibility of JavaScript features.
- Verified consistency in loading and displaying content.

2. Cross-OS Testing:

Testing on different operating systems helps ensure platform independence:

- **Windows:** Verified using Chrome and Firefox, ensuring correct functionality and rendering.
- **macOS:** Tested on Safari and Chrome for UI consistency and performance.
- **Linux:** Tested on Ubuntu using Firefox and Chrome to ensure system compatibility.

Testing Steps:

- Validated functionality such as placing orders, venue bookings, and managing menus.
- Ensured seamless performance across operating systems.



3. Responsive Design Testing (Screen Size):

Ensuring the portal is responsive across different screen sizes is crucial for a good user experience:

- **Mobile Devices:** Tested on screen widths from 320px to 768px (e.g., iPhone, Android phones).
- **Tablets:** Tested on screens between 768px and 1024px.
- **Desktop/Laptop:** Verified functionality and design for widths above 1024px.

Testing Steps:

- Checked whether layouts adapt properly across different devices.
- Ensured that buttons, menus, and forms are accessible and usable on smaller screens.

4. API Testing with Postman:

APIs were tested using **Postman** to ensure that they function correctly and return expected data:

- **User Authentication:** Tested POST /api/users/register and POST /api/users/login for registration and login, verifying JWT token generation.
- **Order Placement:** Used POST /api/orders to place orders, ensuring the correct flow of data from the frontend to the backend.
- **Inventory Management:** Verified GET /api/inventory and POST /api/inventory to ensure that items could be retrieved, added, updated, and deleted.
- **Error Handling:** Deliberately triggered errors (e.g., missing required fields) to test response handling and appropriate error messages.

Testing Steps:

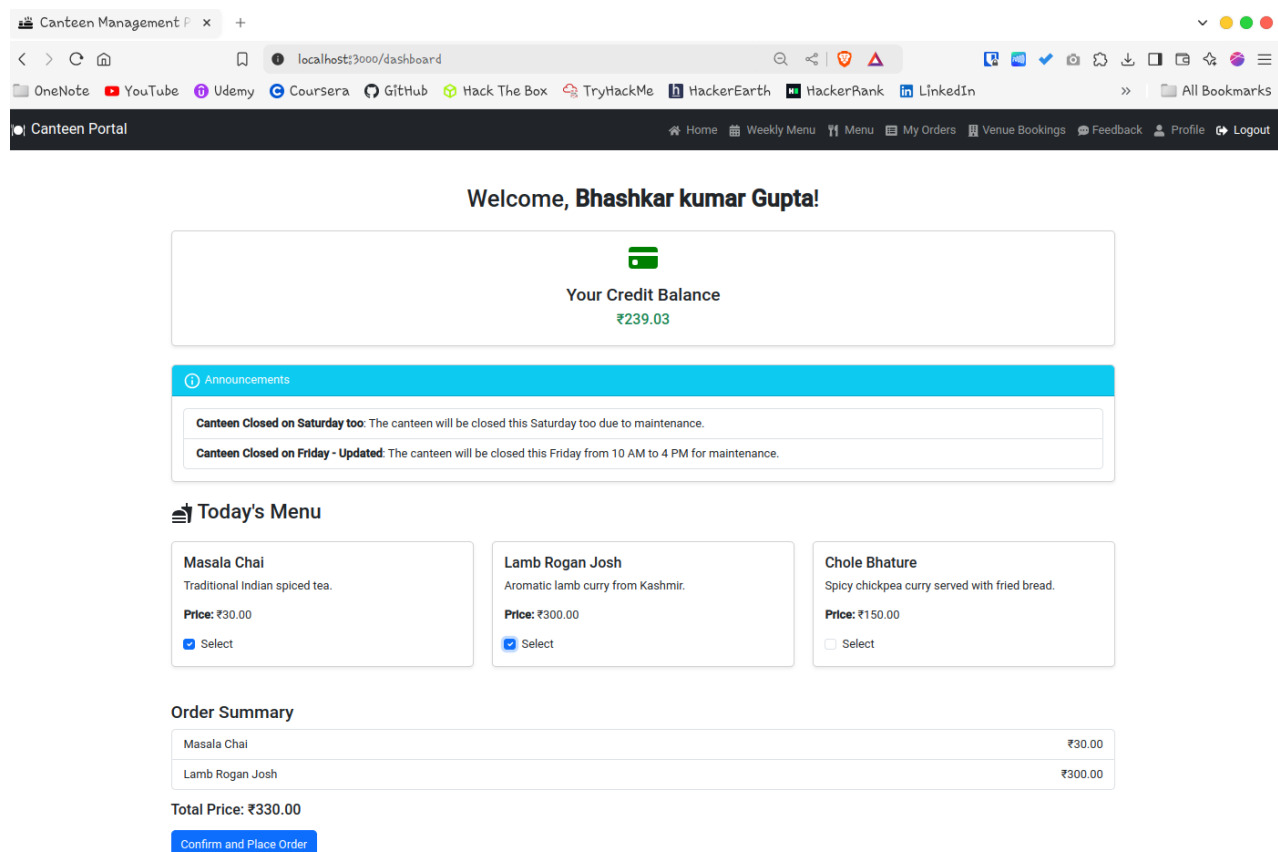
- Validated each API route with correct and incorrect data.
- Checked for correct status codes (200, 400, 401, etc.).
- Ensured proper authentication handling using JWT tokens.



9. UI and their screenshot:

User Interface for Normal Users

The user interface for normal users is designed to be simple, intuitive, and user-friendly, allowing users to easily navigate the portal and access essential features. Upon logging in, users are greeted with a dashboard that provides a personalized overview of their account.



Dashboard Overview:

- **Personalized Welcome:** A greeting message creates a welcoming environment.
- **Credit Balance Display:** Shows the remaining credits users can spend within the canteen, important for tracking spending on orders.
- **Announcements:** Displays important updates like canteen notices, special offers, or events posted by the admin or cook.



- **Today's Menu:** Displays available menu items for the day, including descriptions, prices, and availability, with unavailable items clearly marked.

Available Pages for Users:

Canteen Portal
Home
Weekly Menu
Menu
My Orders
Venue Bookings
Feedback
Profile
Logout

All Menu Items

Chole Bhature
 Spicy chickpea curry served with fried bread.
Price: ₹150.00
 Quantity:

Lamb Rogan Josh
 Aromatic lamb curry from Kashmir.
Price: ₹300.00
 Quantity:

Butter Chicken
 Creamy tomato-based curry with tender chicken pieces.
Price: ₹6.99
 Quantity:

Masala Chai
 Traditional Indian spiced tea.
Price: ₹30.00
 Quantity:

Machali chawal
 Machali and chawal
Price: ₹110.00
 Quantity:

Palak Paneer
 Spinach gravy cooked with cottage cheese cubes.
Price: ₹170.00
 Quantity:

1
2

Order Summary

Chole Bhature (x1)	₹150
Machali chawal (x1)	₹110
Total Price: ₹260.00	

Confirm and Place Order

© 2024 Canteen Management Portal

1. Menu Page:

- Users can explore food and drink options.
- Items are presented with descriptions, prices, and availability.
- Search and filter functionalities help users find specific items, including filters like meal type (e.g., breakfast, lunch) or dietary needs.

Order History

Search Orders

Search by status or item name

Search

Reset

Filter by Date

Start Date

dd/mm/yyyy



End Date

dd/mm/yyyy



Filter

Reset

Order ID	Order Date	Total Amount	Status	Items
17	2024-09-27	₹30.00	accepted	Masala Chai (x1) Rate: Select Rating Submit Rating
16	2024-09-25	₹200.00	accepted	Palak Paneer (x1) Rate: Select Rating Submit Rating
15	2024-09-25	₹80.00	accepted	Gulab Jamun (x1) Rate: Select Rating Submit Rating
14	2024-09-25	₹240.00	pending	Gulab Jamun (x3)
13	2024-09-25	₹206.99	rejected	Butter Chicken (x1) Palak Paneer (x1)
12	2024-09-25	₹236.99	pending	Butter Chicken (x1) Chole Bhature (x1) Gulab Jamun (x1)

2. My Orders Page:

- Displays users' past and current orders, organized with details like items, total price, and order status (approved, rejected, pending).
- Users can rate and provide feedback on their orders.
- Search and filter functions allow users to quickly find specific orders.



Canteen Portal Home Weekly Menu Menu My Orders Venue Bookings Feedback Profile Logout

Venue Bookings

Book a Venue

Select Venue

Booking Date

Start Time

End Time

Purpose

Book Venue

Search My Bookings

Search Reset

Filter by Date

Start Date

End Date

Filter Reset

Venue	Booking Date	Time	Purpose	Status	Admin Comment
Conference Hall A	2024-09-26	07:39:00 - 07:39:00	testing	approved	approved2

3. Venue Booking Page:

- Users can request a venue for events, selecting from options like conference rooms or halls, and specifying the date, time, and purpose.
- Users can check the status of their booking (approved, rejected, pending) and view admin comments.

localhost:3000/feedback

YouTube Udemy Coursera GitHub Hack The Box TryHackMe HackerEarth HackerRank LinkedIn

Home Weekly Menu Menu My Orders Venue Bookings Feedback

💬 Suggestion and Feedback

Submit General Feedback

Submit Feedback

Submit Venue Feedback

Submit Venue Feedback



4. Feedback Page:

- Users can provide feedback on their overall experience, meals, or venues they've booked.
- Encourages users to share suggestions for service improvements.

5. Profile Page:

- Users can update account details, such as contact information, change passwords, and review account history.
- Includes the ability to view credits added by the admin.

User Interface for Cook

The cook's interface is designed to streamline kitchen operations, making it easy to manage orders, inventory, and menus. The layout is simple and efficient, allowing the cook to stay organized while providing accurate information to users.

Cook Dashboard Overview:

- **Welcome Message:** A personalized greeting sets a user-centric tone.



- **Announcements:** Displays important updates from the admin or previous cook notices.
- **Today's Menu:** Shows the current day's menu, allowing the cook to quickly review and adjust items (e.g., marking items out of stock).
- **Pending Orders:** Displays the latest three order requests for quick approval or rejection.
- **Low Inventory Alerts:** Highlights the three items with the lowest stock for efficient inventory tracking and restocking.

Canteen Portal

HomeMenuInventoryOrdersFeedbackProfileLogout

Welcome, Sita Sharma

Announcements

Canteen Closed on Saturday too

The canteen will be closed this Saturday too due to maintenance.

By Alice Smith

Canteen Closed on Friday - Updated

The canteen will be closed this Friday from 10 AM to 4 PM for maintenance.

By Alice Smith

Today's Menu

Masala Chai

Traditional Indian spiced tea.

Price: ₹30.00

Lamb Rogan Josh

Aromatic lamb curry from Kashmir.

Price: ₹300.00

Chole Bhature

Spicy chickpea curry served with fried bread.

Price: ₹150.00

Pending Orders (Oldest 3)

Order #14

Gulab Jamun (x3) - ₹80.00

Total: ₹240.00

AcceptReject

Order #12

Butter Chicken (x1) - ₹6.99

Gulab Jamun (x1) - ₹80.00

Chole Bhature (x1) - ₹150.00

Total: ₹236.99

AcceptReject

Order #11

Palak Paneer (x1) - ₹200.00

Total: ₹200.00

AcceptReject

Low Inventory Items (Lowest 3)

Cooking Oil - 18.95 left

Rice - 41.00 left

Flour - 100.00 left

© 2024 Canteen Management Portal

Cook Pages:

Canteen Portal

HomeMenuInventoryOrdersFeedbackProfileLogout

Weekly Menu Management

View Weekly Menu

Select Day:
Monday

Chole Bhature

₹150.00

Remove

Gulab Jamun

₹50.00

Remove

Add Menu Items to Monday

Search for a menu item...

Chole Bhature

₹150.00

Add

Lamb Rogan Josh

₹300.00

Add

Butter Chicken

₹6.99

Add

Masala Chai

₹30.00

Add

12

Create New Menu Item

Name

Description

Price

Availability

Available

Category

Vegetarian

Create Menu Item

Edit/Delete Menu Items

Search for a menu item...

Chole Bhature

Description: Spicy chickpea curry served with fried bread.

Price: ₹150.00

Category: vegetarian

Availability: Available

EditDelete

Lamb Rogan Josh

Description: Aromatic lamb curry from Kashmir.

Price: ₹300.00

Category: non-vegetarian

Availability: Available

EditDelete

Butter Chicken

Description: Creamy tomato-based curry with tender chicken pieces.

Price: ₹6.99

Category: non-vegetarian

Availability: Available

EditDelete

Masala Chai

Description: Traditional Indian spiced tea.

Price: ₹30.00

Category: beverages

Availability: Available

EditDelete

Machali chawal

Description: Machali and chawal

Price: ₹110.00

Category: non-vegetarian

Availability: Available

EditDelete

Palak Paneer

Description: Spinach gravy cooked with cottage cheese cubes.

Price: ₹170.00

Category: vegetarian

Availability: Available

EditDelete

12

1. Menu Management Page:

- Allows the cook to create, edit, or delete menu items.
- Key details include name, description, price, category, and availability.
- The cook can also plan menus for the entire week, ensuring users know what's available in advance.

Canteen Portal

HomeMenuInventoryOrdersFeedbackProfileLogout

Inventory Management

Search for inventory items...

Rice

Quantity: 41.00

Type: ingredient

Comments: Used 5 kg for today'... Show More

EditDelete

Flour

Quantity: 100.00

Type: Ingredients

Comments: Basmati rice

EditDelete

Plates

Quantity: 100.00

Type: utensil

Comments: Stainless steel plat... Show More

EditDelete

Cooking Oil

Quantity: 18.95

Type: ingredient

Comments: Sunflower oil, 20 li... Show More

EditDelete

1

Add New Inventory Item

Item Name

Quantity

Item Type

Comments

Add Inventory Item

Add Recipe to Menu Items

Search for menu items...

Chole Bhature

Spicy chickpea curry served with fried bread.

Add Recipe

Lamb Rogan Josh

Aromatic lamb curry from Kashmir.

Add Recipe

Butter Chicken

Creamy tomato-based curry with tender chicken pieces.

Add Recipe

1

2

3

© 2024 Canteen Management Portal

2. Inventory Management Page:

- The cook can add, edit, or delete inventory items.
- Inventory is linked to menu items, and stock is automatically reduced when users place orders.
- Real-time tracking ensures the cook can manage stock levels effectively.

Canteen Portal
Home
Menu
Inventory
Orders
Feedback
Profile
Logout

Order Management

Pending (8)

Accepted (8)

Rejected (1)

Order ID: 17

User: Bhashkar kumar Gupta

Email: bhashkar@example.com

Contact: 993455998908

Status: accepted

Total Amount: ₹30.00

Date: 27/09/2024

Time: 00:35:14

Masala Chai | Quantity: 1 | Price: ₹30.00

✓ Approve

✗ Reject

Order ID: 16

User: Bhashkar kumar Gupta

Email: bhashkar@example.com

Contact: 993455998908

Status: accepted

Total Amount: ₹200.00

Date: 25/09/2024

Time: 12:57:45

Palak Paneer | Quantity: 1 | Price: ₹200.00

✓ Approve

✗ Reject

Order ID: 15

User: Bhashkar kumar Gupta

Email: bhashkar@example.com

Contact: 993455998908

Status: accepted

Total Amount: ₹80.00

Date: 25/09/2024

Time: 05:03:29

Gulab Jamun | Quantity: 1 | Price: ₹80.00

✓ Approve

✗ Reject

Order ID: 14

User: Bhashkar kumar Gupta

Email: bhashkar@example.com

Contact: 993455998908

Status: pending

Total Amount: ₹240.00

Date: 25/09/2024

Time: 04:20:37

Gulab Jamun | Quantity: 3 | Price: ₹80.00

✓ Approve

✗ Reject

Order ID: 13

User: Bhashkar kumar Gupta

Email: bhashkar@example.com

Contact: 993455998908

Status: rejected

Total Amount: ₹206.99

Date: 25/09/2024

Time: 04:20:06

Butter Chicken | Quantity: 1 | Price: ₹6.99

Palak Paneer | Quantity: 1 | Price: ₹200.00

✓ Approve

✗ Reject

Order ID: 12

User: Bhashkar kumar Gupta

Email: bhashkar@example.com

Contact: 993455998908

Status: pending

Total Amount: ₹236.99

Date: 25/09/2024

Time: 04:05:46

Butter Chicken | Quantity: 1 | Price: ₹6.99

Gulab Jamun | Quantity: 1 | Price: ₹80.00

Chole Bhature | Quantity: 1 | Price: ₹150.00

✓ Approve

✗ Reject

1
2
3

© 2024 Canteen Management Portal

3. Order Management Page:

- Displays all orders with user details and item information.
- The cook can approve or reject orders based on availability.



- Search and filter functionalities make it easy to find specific orders or filter by date and status.

Canteen Portal

[Home](#) [Menu](#) [Inventory](#) [Orders](#) [Feedback](#) [Profile](#) [Logout](#)

Feedback Management

General Feedback

Search by User Name or Email

Filter by Date

Apply Filter Clear Filter

Bhashkar kumar Gupta
bhashkar@example.com
The canteen management portal should have more features
2024-09-26

Bhashkar kumar Gupta
bhashkar@example.com
This is general suggestion feedback
2024-09-25

Alice Smith
alice@example.com
The butter chicken was delicious and the service was excellent!
2024-09-24

Alice Smith
alice@example.com
The butter chicken was delicious and the service was excellent!
2024-09-24

Sita Sharma
cook@example.com
The butter chicken was delicious and the service was excellent!
2024-09-24

1

Venue Feedback

Search by Venue Name, User Name, or Email

Filter by Date

Apply Filter Clear Filter

Swimming Pool
Bhashkar kumar Gupta - bhashkar@example.com
Swimming pool should have mermaid in it
2024-09-26
Delete

Outdoor Garden
Bhashkar kumar Gupta - bhashkar@example.com
Garden is green
2024-09-25
Delete

Conference Hall A
Alice Smith - alice@example.com
It was cool
2024-09-24
Delete

1

© 2024 Canteen Management Portal

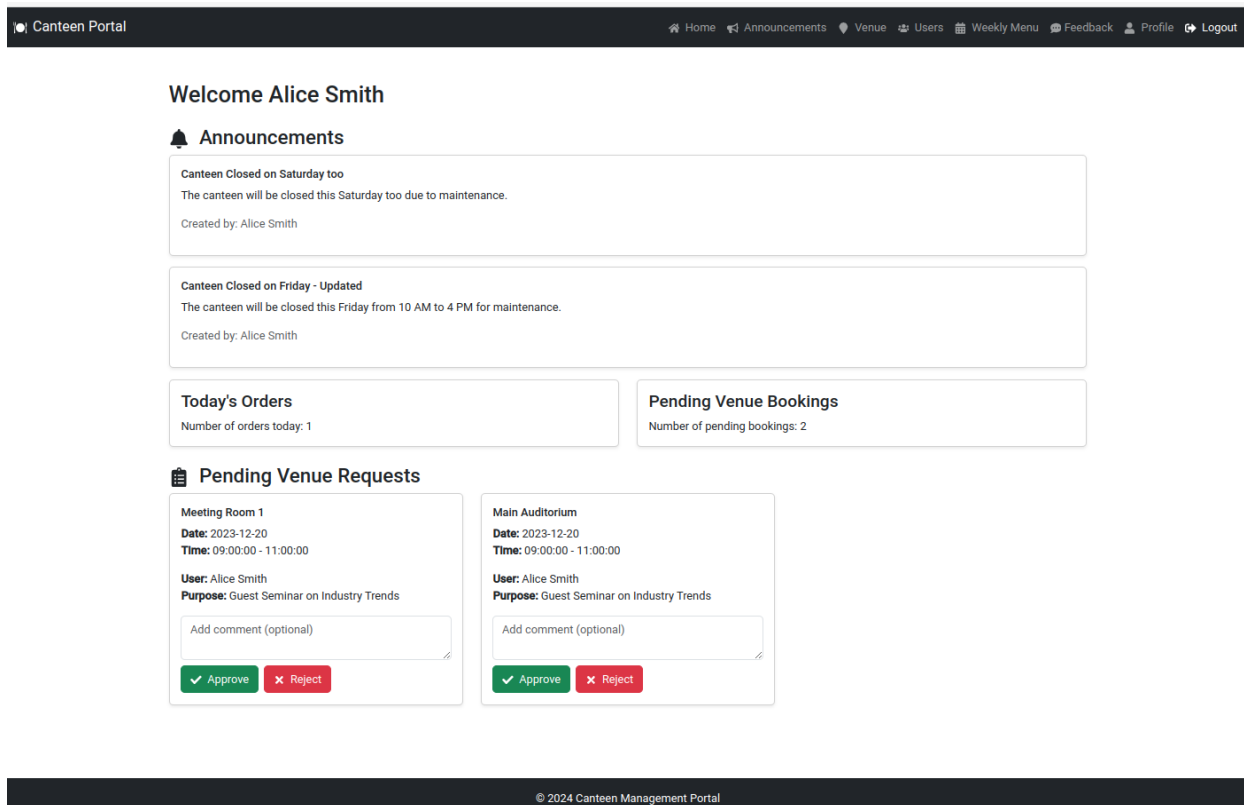
4. Feedback Page:

- The cook can view user feedback on orders or general experiences.
- Feedback is critical for improving dishes or the overall service.



User Interface for Admin

The admin interface is built to efficiently manage canteen operations, focusing on user management, venue bookings, announcements, and feedback. It offers a streamlined layout for handling these critical tasks.



The screenshot displays the 'Canteen Portal' admin interface. At the top, a dark navigation bar includes the portal name and links for Home, Announcements, Venue, Users, Weekly Menu, Feedback, Profile, and Logout. The main content area is titled 'Welcome Alice Smith'. Below this, the 'Announcements' section shows two notices: 'Canteen Closed on Saturday too' and 'Canteen Closed on Friday - Updated', both created by Alice Smith. The 'Today's Orders' section indicates 1 order, while 'Pending Venue Bookings' shows 2 bookings. The 'Pending Venue Requests' section features two request cards. The first card is for 'Meeting Room 1' on 2023-12-20 from 09:00:00 to 11:00:00, requested by Alice Smith for a 'Guest Seminar on Industry Trends'. The second card is for the 'Main Auditorium' on the same date and time, also requested by Alice Smith for the same purpose. Both cards include a text area for optional comments and 'Approve' (green) and 'Reject' (red) buttons. A footer bar at the bottom states '© 2024 Canteen Management Portal'.

Admin Dashboard Overview:

- **Welcome Message:** Personalized greeting to enhance user experience.
- **Announcements Section:** Displays current announcements, keeping the admin updated on important notices.
- **Orders Overview:** Shows the number of orders placed for the day, offering insight into daily operations.
- **Pending Venue Requests:** Displays the latest three venue requests for approval or rejection, with the ability to add comments for users.

Admin Pages:



Canteen Portal

Home Announcements Venue Users Weekly Menu Feedback Profile Logout

Announcement Management

Search announcements...

Create Announcement

Canteen Closed on Saturday too
The canteen will be closed this Saturday too due to maintenance...[Show more](#)
Creator: Alice Smith
Created At: 24/09/2024
Updated At: 24/09/2024
[Edit](#) [Delete](#)

Canteen Closed on Friday - Updated
The canteen will be closed this Friday from 10 AM to 4 PM for maintenance...[Show more](#)
Creator: Alice Smith
Created At: 24/09/2024
Updated At: 24/09/2024
[Edit](#) [Delete](#)

1

1. Announcements Page:

- Create, edit, and delete announcements visible to users and cooks.
- Search functionality for managing multiple announcements efficiently.

Canteen Portal

Home Announcements Venue Users Weekly Menu Feedback Profile Logout

Venue Management

Venue Requests

Search by user, venue name, or booking ID

Filter by date [Clear Date Filter](#)

[Approved \(1\)](#) [Pending \(2\)](#) [Rejected \(2\)](#)

Booking ID: 3
Venue: Meeting Room 1 (Second Floor, Building C)
Booking Date: 20/12/2023
Start Time: 09:00:00
End Time: 11:00:00
Status: pending
[Approve](#) [Reject](#)

Booking ID: 4
Venue: Main Auditorium (Ground Floor, Building A)
Booking Date: 20/12/2023
Start Time: 09:00:00
End Time: 11:00:00
Status: pending
[Approve](#) [Reject](#)

Booking ID: 2
Venue: Main Auditorium (Ground Floor, Building A)
Booking Date: 16/12/2023
Start Time: 14:00:00
End Time: 17:00:00
Status: rejected
[Approve](#) [Reject](#)

Booking ID: 5
Venue: Conference Hall A (First Floor, Building B)
Booking Date: 26/09/2024
Start Time: 07:39:00
End Time: 07:39:00
Status: approved
[Approve](#) [Reject](#)

< 1 >

Venues



Add New Venue

Venue Name

Location

Description

Add Venue

2. Venue Management Page:

- Manage venue details like name, capacity, and availability.
- Approve or reject booking requests, with search and filter options for efficient management.

User Management

Create New User

Name

Employee ID

Email

Password

Address

Contact Number

Role

Create User

Reset User Password

Add Credit

3. User Management Page:

- Create new users, reset passwords, and add credits to accounts.
- Search and filter options for quick user profile management.



Feedback Management

General Feedback

Search by User Name or Email

Filter by Date

[Apply Filter](#)
[Clear Filter](#)

Bhashkar kumar Gupta
bhashkar@example.com

The canteen management portal should have more features

2024-09-26

Bhashkar kumar Gupta
bhashkar@example.com

This is general suggestion feedback

2024-09-25

Alice Smith
alice@example.com

The butter chicken was delicious and the service was excellent!

2024-09-24

Alice Smith
alice@example.com

The butter chicken was delicious and the service was excellent!

2024-09-24

Sita Sharma
cook@example.com

The butter chicken was delicious and the service was excellent!

2024-09-24

1

Venue Feedback

Search by Venue Name, User Name, or Email

Filter by Date

[Apply Filter](#)
[Clear Filter](#)

Swimming Pool

Bhashkar kumar Gupta - bhashkar@example.com
Swimming pool should have mermaid in it

2024-09-26

Delete

Outdoor Garden

Bhashkar kumar Gupta - bhashkar@example.com
Garden is green

2024-09-25

Delete

Conference Hall A

Alice Smith - alice@example.com
It was cool

2024-09-24

Delete

1

4. Feedback Management Page:

- Monitor and manage user feedback to ensure continuous improvement.
- Search and filter functionality to easily navigate feedback entries, with the option to delete inappropriate feedback.



10. Security Implementation:

The project incorporates several security measures to ensure data protection and secure access to the system:

1. **bcryptjs:**

- Used for securely hashing user passwords before storing them in the database. Passwords are verified using bcrypt hashes during login, preventing plaintext storage.

2. **jsonwebtoken (JWT):**

- Implements secure authentication by generating JSON Web Tokens (JWT) upon user login. JWTs are used to protect routes and ensure only authenticated users can access certain features.

3. **helmet:**

- Adds security-related HTTP headers to protect against common web vulnerabilities like cross-site scripting (XSS) and clickjacking.

4. **express-rate-limit:**

- Limits the number of requests a user can make within a specific time frame, protecting the application from brute force and denial-of-service (DoS) attacks.

5. **Access Control (ACL):**

- Role-based access control ensures that only users with appropriate roles (admin, cook, or user) can access specific features. This prevents unauthorized access to sensitive operations like menu management or venue bookings.

These security layers collectively help protect the system from vulnerabilities, unauthorized access, and attacks.



11. Limitations:

- **Limited Scalability:** The system may struggle to handle a large number of concurrent users due to the absence of load balancing or horizontal scaling mechanisms.
- **Role-based Restrictions:** The role-based access control is predefined, limiting flexibility if additional roles or custom permissions are needed in the future.
- **No Notification System:** There is no built-in notification system to alert users or admins about important events like order status updates or low inventory.
- **No Multi-Factor Authentication (MFA):** The system relies solely on password-based authentication, which could be strengthened with multi-factor authentication.
- **Not a Native App:** The system isn't developed as a native mobile app but can be used as a Progressive Web App (PWA) for a similar experience.
- **No Food Item Image Support:** Menu items currently lack the ability to display images, which could improve user experience.
- **No Payment Integration:** External payment gateways are not integrated, which may be necessary in a real-world deployment.

12. Future Scope:

- **Scalability Enhancements:** Implementing load balancing, database optimization, and cloud infrastructure will allow the system to handle a larger number of concurrent users efficiently.
- **Notification System:** Adding a notification system (via email, SMS, or in-app notifications) to alert users and admins about important events such as order updates, booking confirmations, or low inventory alerts.
- **Multi-Factor Authentication (MFA):** Introducing multi-factor authentication will enhance security by requiring an additional layer of verification during user login.
- **Native Mobile App:** Developing native apps for iOS and Android would improve the mobile user experience, although the PWA (Progressive Web App) version currently provides basic mobile functionality.
- **Image Support for Menu Items:** Incorporating images for menu items will make the menu more visually appealing and improve user decision-making.



- **Payment Gateway Integration:** Adding support for payment gateways will allow users to make direct online payments, making the system more practical for real-world deployments.
- **Advanced Analytics:** Adding analytics for tracking user behaviour, popular menu items, and peak order times could help admins and cooks optimize operations and menu planning.
- **Custom Roles and Permissions:** Expanding the role-based access control to allow the creation of custom roles and permissions for more flexible management.

12. Bibliography:

- **Udemy Course:** *The Complete Web Development Bootcamp*
<https://www.udemy.com/course/the-complete-web-development-bootcamp>
- **Node.js Documentation:**
Official documentation used for backend development and server setup.
<https://nodejs.org/en/docs/>
- **React.js Documentation:**
Official documentation for building the frontend using React.js.
<https://reactjs.org/docs/getting-started.html>
- **Node Modules Documentation:**
Documentation for various Node.js modules like bcryptjs, express-rate-limit, helmet, and jsonwebtoken.
- **Google:**
Used for research, finding relevant information, and solving development issues.
- **Stack Overflow and Reddit:**
Platforms used for debugging and troubleshooting errors during development.
- **Mohit Sharma:**
Project guide and mentor.
- **Apurva Rani :**
Provided project requirements and user interface design ideas.

